

Desenvolvimento de Árvore-B em Memória Secundária

Trabalho de implementação e análise de desempenho

Caio Uehara Yuri Calzzani Igor Felipe

PPGCA — DCM

Algoritmos e Estruturas de Dados

Docente: Prof. Dr. José Augusto Baranauskas

Agenda

Decisões e Arquitetura

Demonstração

Análise Experimental

Discussão

Decisões e Arquitetura

Decisões de Implementação

Armazenamento da raiz

Mantida de forma **dupla**:

- em memória, no atributo `rootID`;
- em disco, no registro 0 (cabeçalho).

O cabeçalho é reescrito em 4 momentos: *criação*, *split*, *colapso* da raiz e *carregamento*.

Reaproveitamento de nós — sim

- **Estrutura:** lista de livres (“lixeira”), topo no `A[1]` do cabeçalho, encadeada pelo `A[0]` de cada registro livre.
- **Como:** `freeNode()` empilha o nó removido; `allocateNode()` recicla antes de crescer o arquivo.

Armazenamento estritamente em memória secundária

Cada nó é um registro de **tamanho fixo** $(2M + 3) \cdot \text{sizeof}(\text{int})$, lido/escrito **um por vez** por *seek + struct* (sem cache de vários nós em RAM) — atende ao requisito do enunciado.

```
template <int M>
struct BTreeNode { int numKeys; int K[M+1]; int A[M+1]; };

file.seekg(id * sizeof(BTreeNode<M>), std::ios::beg); // 1 no
file.read ((char*)&node, sizeof(BTreeNode<M>));      // 1 registro
```

Métodos Implementados

BTree<M>

search, insert, split, remove,
removeRecursive, borrowFromLeft/Right,
mergeChildren, loadFromFile, getHeight.

BTreeNode

findIndex(key) localiza a chave ou o ponto de
inserção.

DiskManager (contador de I/O)

incrementRead/Write,
getReadCount/WriteCount, resetCounters,
getFileSize.

Módulo Display

displayTree, displayLevelOrder, dumpFile,
displaySearchResult. (*lógica e visualização
separadas.*)

Demonstração

Caso de uso em código: reaproveitamento de nós

A “lixeria” do Slide 2 são estes dois métodos (src/btree.h):

```
// Aloca um nó: recicla da lixeira antes de crescer o arquivo.
int allocateNode() {
    BTreeNode<M> hdr; readNode(0, hdr);
    if (reuseEnabled && hdr.A[1] != 0) {          // há nó livre?
        int id = hdr.A[1];                        // topo da lixeira
        BTreeNode<M> n; readNode(id, n);
        hdr.A[1] = n.A[0];                        // desempilha
        writeNode(0, hdr);
        return id;                                // reaproveita o registro
    }
    file.seekp(0, std::ios::end);                  // senão, cresce o .dat
    return (int)(file.tellp() / sizeof(BTreeNode<M>));
}
// Libera um nó: empilha-o na lixeira (encadeada pelo A[0]).
void freeNode(int id) {
    BTreeNode<M> hdr; readNode(0, hdr);
    BTreeNode<M> dead; dead.A[0] = hdr.A[1];      // aponta p/ topo antigo
    writeNode(id, dead);
    hdr.A[1] = id; writeNode(0, hdr);              // novo topo da lixeira
}
```


Caso de uso em código: *split*

Quando um nó enche ($\text{numKeys} == M$), ele é dividido e a mediana sobe:

```
void split(std::vector<int>& path) {
    int mid = (M+1)/2, leftID = path.back(); path.pop_back();
    BTreeNode<M> L; readNode(leftID, L);
    int rightID = allocateNode(); BTreeNode<M> R;
    int promo = L.K[mid]; // mediana promovida
    R.A[0] = L.A[mid]; R.numKeys = M - mid;
    for (int k=1; k<=M-mid; k++){ R.K[k]=L.K[mid+k]; R.A[k]=L.A[mid+k]; }
    L.numKeys = mid - 1; // L fica com a metade inferior
    writeNode(leftID, L); writeNode(rightID, R);
    if (path.empty()) { // raiz dividida -> nova raiz
        /* aloca nova raiz com 'promo' e atualiza o cabeçalho (rec 0) */
    } else { // insere 'promo' no pai...
        /* ...e, se o pai encher, split(path) recursivo (cascata) */
    }
}
```

A interface interativa em ação

Saída **real** do ./main.exe ($M=3$; inseridas 30, 20, 40, 55, 10, 35, 50; opção 4 – *Imprimir árvore*):

```
=====
                        ARVORE B
=====
1. Inserir chave
2. Remover chave
3. Buscar chave
4. Imprimir arvore
5. Estatisticas de I/O
6. Dump bruto do arquivo .dat
0. Voltar / Sair
Escolha uma opcao:

+===== RESULTADO =====+
--- Arvore (indentada) ---
[30 40] (rec 3)
  [10 20] (rec 1)
  [35] (rec 2)
  [50 55] (rec 4)

--- Arvore (por nivel) ---
Nivel 0: [30 40] (rec 3)
Nivel 1: [10 20] (rec 1)  [35] (rec 2)  [50 55] (rec 4)
+=====+
```

Estado em disco: estatísticas e *dump* bruto

Contadores de I/O da sessão:

```
+===== RESULTADO =====+  
--- Estatísticas do Disco ---  
Leituras (I/O): 57  
Escritas (I/O): 16  
Tamanho do arq: 180 bytes  
+=====+
```

Registros físicos do .dat:

```
+===== RESULTADO =====+  
--- Dump bruto do .dat (4 registros, 144 bytes, sizeof(no)=36) ---  
raiz: rec 3  
rec 0 (cabecalho): numKeys=0 | A[0]=3  
rec 1: numKeys=2 | A[0]=0 K[1]=10 A[1]=0 K[2]=20 A[2]=0  
rec 2: numKeys=2 | A[0]=0 K[1]=40 A[1]=0 K[2]=55 A[2]=0  
rec 3: numKeys=1 | A[0]=1 K[1]=30 A[1]=2  
+=====+
```

O rec 0 é o cabeçalho: A[0] guarda a raiz (aqui rec 3) e A[1], o topo da lixeira. 1 registro = 1 nó (144 bytes ÷ 36 = 4 registros).

Compilação e execução: o Makefile

Todo o fluxo do projeto é automatizado por alvos do make:

`make` compila `main.exe` (alvo padrão, `g++ -std=c++17`).

`make run` garante `tmp/` e abre o `modo interativo`.

`make test` compila e roda os `27 testes` (`tests/test_btree.cpp`).

`make run_experiments` roda a bateria completa $\rightarrow$ `data/*.csv`.

`make graficos` gera as figuras da análise (`slides/img/*.png`).

`make sessions` captura os *prints* reais da interface (`slides/sessions/*.txt`).

`make slides-assets` = `graficos` + `sessions` (regenera o que o deck inclui).

`make slides` compila `slides/apresentacao.pdf` (`pdflatex`, 2 passadas).

`make clean` remove binários, `tmp/` e auxiliares do $\text{\LaTeX}$.

Análise Experimental

Testes de validação

27 casos (11 search, 7 insert, 9 remove), cada um em processo filho (fork) para isolar *crashes*.

Além disso, um experimento de *churn* (insere $N \rightarrow$ remove $N/2 \rightarrow$ reinsere $N/2$) compara a ocupação do arquivo **com e sem** reaproveitamento de nós.

Desempenho — 70 execuções

- Ordens M : 3, 5, 10, 50, 100, 500, 1000
- Volumes: 10^3 , 10^4 , $5 \cdot 10^4$, 10^5 , 10^6
- Padrões: Sequencial e Aleatório

Testes de validação: por *fixtures*

- Cada caso roda em um **processo filho** (fork): um *crash* (ex.: split com bug) vira um **único FAIL** em vez de abortar a suíte.
- **27 casos** cobrindo: search (presente/ausente/vazia), insert (sem split, split, *cascata*, $m=5$), remove (folha, *borrow*, *merge*, colapso da raiz, no-op).
- **Fixtures** em tests/fixtures/*.txt: árvores em texto legível (com comentários) $\Rightarrow$ fáceis de inspecionar e versionar.

Formato da fixture (search_tree.txt):

```
root 1
1  1  2  30 3          # rec1: A0=2 K1=30
      A1=3
2  1  4  20 5
3  1  6  40 7
4  2  0  10 0  15 0  # folha [10 15]
...
```

Linha: id numKeys A[0] (K[i] A[i])...

Fluxo de um caso: loadFromFile → operação → compareLogical

1. loadFromFile **materializa** a fixture (texto) no arquivo binário .dat — *before* e *after*.
2. A operação (insert/remove) roda no fork sobre o *before*; o destrutor faz *flush* antes do pai ler.
3. compareLogical compara o resultado com o *after*.

Comparação lógica: percorre as duas árvores em paralelo e confere *forma + chaves*, **ignorando** ids físicos e nós mortos da lixeira — um remove pode deixar o arquivo com layout diferente, mas a árvore é a mesma.

```
pid = fork();
if (pid == 0) {                                // filho
    BTree<M> t(actual);
    t.loadFromFile(before);
    op(t);                                     // insert/
        remove
    _exit(0);                                  // dtor: flush
}
waitpid(pid, &status, 0);
if (WIFSIGNALED(status)) // crashou?
    check(name, false, "crash");
bool ok = compareLogical<M>(
    actual, expected, msg);
check(name, ok, msg);
```


Forçando o disco real: cgroup na análise

Problema: o `.dat` (≤ 36 MB) cabe na RAM, então o `page cache do SO` serve o I/O e o `tempo_io` mede memória, não o disco.

Solução: rodar cada experimento dentro de um `cgroup` de memória limitada, via `systemd-run -user (cgroup v2, sem root)`:

```
systemd-run --user --scope -p MemoryHigh=8M -p MemoryMax=64M \  
    ./main.exe --experiment <M> 1000000 <1=seq|0=rand>
```

Por que MemoryHigh (e não Max)

Limite `soft`: estrangula e força *eviction* do cache (8 MB < arquivo) $\Rightarrow$ os acessos batem no NVMe, mas `não mata` o processo. `MemoryMax` sozinho dá *OOM-kill* sob páginas sujas; aqui ele é só um teto de segurança (64 MB).

O que isso valida

- `reads/writes/bytes/altura` `idênticos` cacheado vs. restrito.
- Só o `tempo` muda — e só no aleatório.
- Saída: `data/resultados_io_real.csv` (coluna modo).

Inserção sequencial × aleatória: o que muda

No experimento, a única diferença é a *ordem das chaves* inseridas (src/main.cpp):

```
int key = sequential ? i : rand(); // Seq: 0,1,2,...   Rand: rand()
tree.insert(key);
```

Sequencial (0, 1, 2, ...)

- Cada chave é a maior $\Rightarrow$ desce sempre para a *folha mais à direita*.
- Toca sempre o mesmo caminho (a *espinha direita*, $\sim$ altura nós) $\Rightarrow$ *alta localidade*.
- Folhas à esquerda “congelam” meio cheias $\Rightarrow$ ocupação $\sim 50\%$ $\Rightarrow$ arquivo *maior*.

Aleatório (rand())

- Cada chave cai em qualquer ponto $\Rightarrow$ desce um *caminho diferente* a cada inserção.
- Toca a *árvore inteira* (11–36 MB) $\Rightarrow$ *baixa localidade*.
- Folhas enchem uniformemente $\Rightarrow$ ocupação $\sim 69\%$ ($\ln 2$) $\Rightarrow$ arquivo *menor*.

É a ordem das *chaves* (não offsets no arquivo): ambos usam seek por id. A ordem é que induz acesso aos nós com alta ou baixa *localização das referências* (AED-01).

Métricas Utilizadas

`M / numKeys / tipo` Ordem, nº de chaves e padrão (Seq/Rand).

`reads / writes` Total de `leituras` e de `escritas` de nós em disco (`readNode/writeNode`), contadas *separadamente*.

`tempo_io` Tempo gasto nas chamadas de I/O.

`bytes` Tamanho final do `.dat` (`getFileSize()`) — ocupação.

`altura` Nº de níveis da árvore (`getHeight()`, descendo 1 nó por vez).

Obs.: `reads/writes` contam também o acesso ao registro 0 (cabeçalho/lixreira) feito a cada alocação.

Tabela de Resultados

Inserção sequencial — data/resultados_experimentos.csv

M	N	altura	reads	writes	bytes
3	10^3	9	10 952	3 964	35 820
3	10^5	16	1 768 897	399 950	3 599 820
3	10^6	19	20 951 387	3 999 941	35 999 784
100	10^6	4	3 803 513	1 061 213	16 571 296
1000	10^6	3	2 504 506	1 006 006	16 064 060

- $M \uparrow \Rightarrow$ altura $\downarrow$ e menos leituras/escritas totais (menos nós tocados por inserção).
- reads $\gg$ writes: cada inserção lê o caminho inteiro, mas só reescreve os nós alterados.

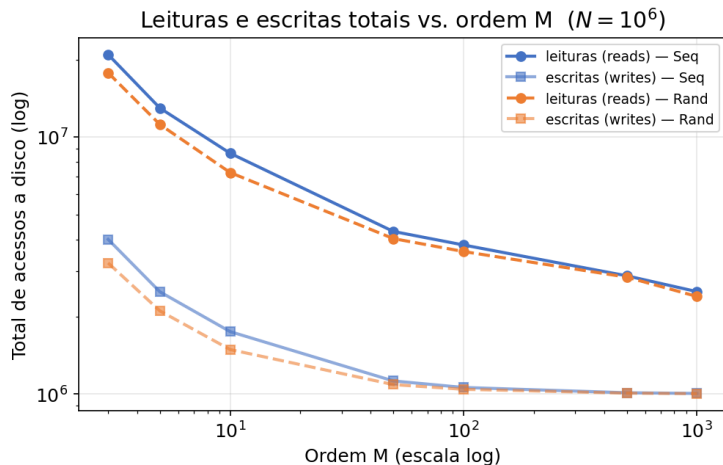
Impacto do Reuso na Ocupação

Workload: insere $N \rightarrow$ remove $N/2 \rightarrow$ reinsere $N/2$ (Seq) — data/reuso_comparacao.csv

M	N	bytes <i>com</i> reuso	bytes <i>sem</i> reuso	economia
3	10^4	359 856	539 784	$\approx 33\%$
10	10^5	2 299 632	3 449 540	$\approx 33\%$
100	10^5	1 657 292	2 485 532	$\approx 33\%$

- Com reuso, a reinserção recicla os nós liberados; sem reuso, o arquivo cresce sempre $\Rightarrow \approx 1/3$ de economia de disco.

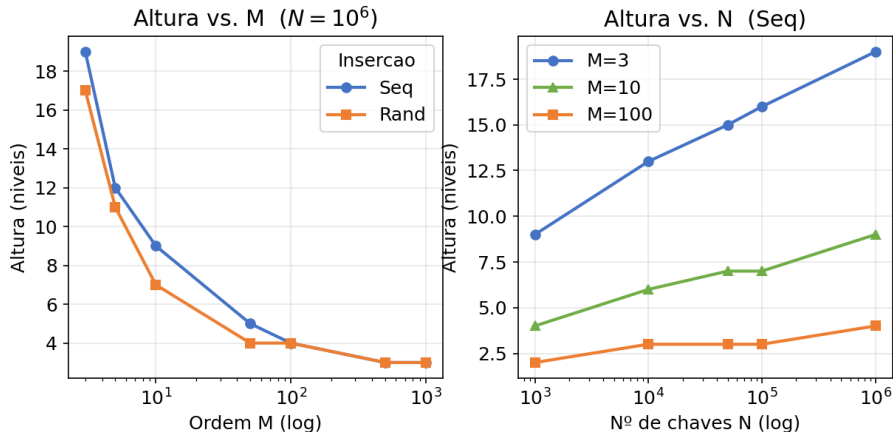
Leituras e escritas $\times$ ordem M



$M \uparrow \Rightarrow$ menos reads e writes totais (árvore mais rasa: menos nós por inserção). As reads caem de ~ 21 para $\sim 2,5$ milhões; reads sempre $>$ writes.

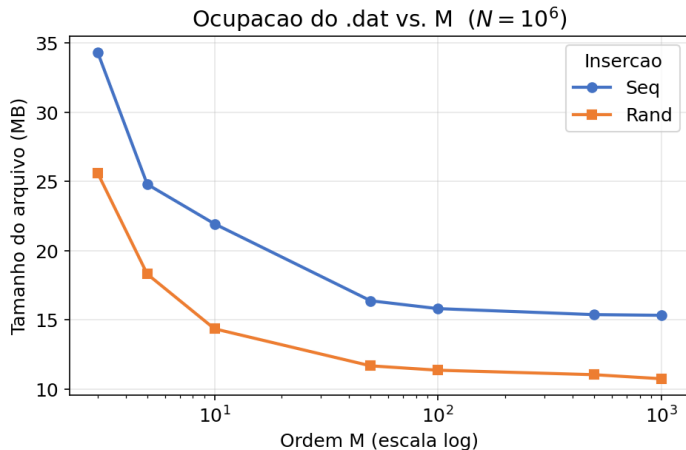
Altura da árvore ($\times M$ e $\times N$)

Altura da árvore: cresce com N , encolhe com M



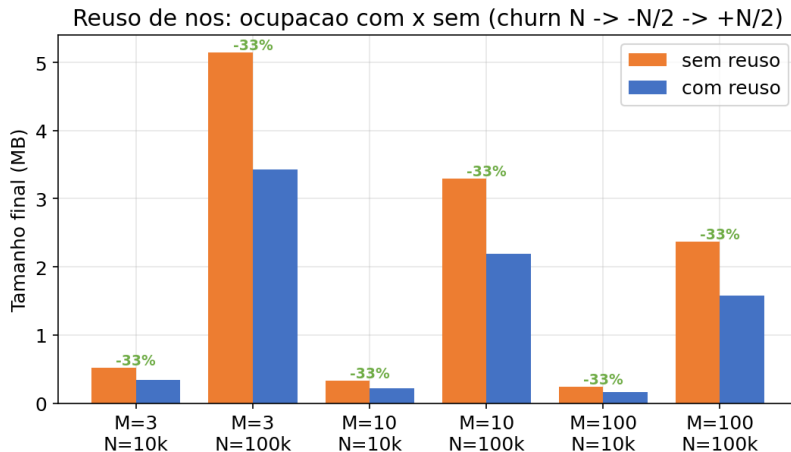
$\text{Altura} \propto \log_M N$: cresce devagar com N e cai bruscamente com M (de 19 níveis em $M=3$ para 3–4 em $M \geq 100$, $N=10^6$).

Ocupação e padrão de inserção (Seq × Rand)



A inserção **aleatória** preenche melhor os nós (arquivo menor); a **sequencial** deixa nós meio-cheios após cada split.

Reuso de nós: ocupação *com* × *sem*



Workload de *churn* (insere N , remove $N/2$, reinsere $N/2$): reciclar nós liberados economiza $\approx 1/3$ de disco.

Discussão

Usamos um assistente de IA como [apoio](#), sempre com validação humana e conferência contra os dados e o enunciado. Onde ajudou:

[Revisão de código](#) Revisão crítica que apontou casos de borda — destaque para a [falha silenciosa de I/O](#) quando `tmp/` não existia.

[Geração de casos de teste](#) Sugestão de *fixtures* e cenários que ampliaram a cobertura para os 27 casos.

[Comentários e documentação](#) Apoio na redação de comentários do código, do README e dos cabeçalhos das funções.

[Revisão dos slides](#) Conferência da aderência ao modelo do enunciado e da consistência dos números com os CSVs.

Conclusão e Referências

- **Ordem M reduz acessos e altura:** para 10^6 chaves, as leituras caem de ~ 21 milhões ($M=3$) para $\sim 2,5$ milhões ($M=1000$) e as escritas de $\sim 4,0$ para $\sim 1,0$ milhão; a altura, de 19 para 3 níveis.
- **Cache vs. disco real:** os tempos da bateria refletem o page cache (o .dat cabe na RAM; device NVMe). Forçando I/O real por cgroup, só a inserção aleatória (baixa localidade) paga disco; a contagem de acessos (lógica) independe da máquina.
- **Reuso de nós economiza $\approx 33\%$ de ocupação do arquivo.**
- **Conformidade:** I/O estritamente 1 nó por vez (1 seek + 1 struct).

Referências: CORMEN et al. *Introduction to Algorithms*, 4. ed., MIT Press, 2022 (Cap. 18). COMER, D. "The Ubiquitous B-Tree". *ACM Comp. Surveys*, 1979. KNUTH, D. *TAOCP*, v.3.

Obrigado!

Caio Uehara • Yuri Calzzani • Igor Felipe